1. Project Group Number/Name

Group 1

---

2. Project Topic/Name

Dynamic Pricing Engine for Multi-Channel DTC Retail

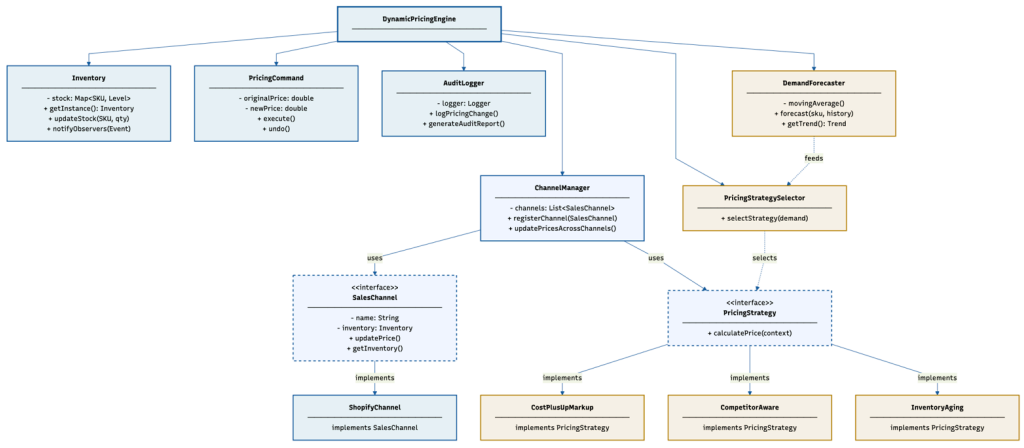
---

3. Problem Statement

Direct-to-consumer (DTC) brands operating across multiple sales channels (Shopify, own website, social commerce) struggle with manual, inconsistent pricing that doesn't adapt to market demand. Our project addresses this by building a unified Dynamic Pricing Engine that forecasts sales demand in real-time, automatically selects the optimal pricing strategy based on market conditions, and synchronizes prices across channels with full audit logging. The system demonstrates how Gang of Four design patterns solve real-world inventory and pricing problems while incorporating lightweight machine learning for demand-driven decision making. By combining predictive analytics with proven design patterns, we create a modular, extensible platform that DTC brands can use to optimize revenue while respecting brand positioning constraints.

---

4. UML Diagram



---

5. Design Patterns to Be Implemented

Pattern	Type	Used For

Singleton	Creational	Single shared Inventory instance across all channels
Strategy	Behavioral	Interchangeable pricing algorithms (cost-plus, competitor-aware, inventory-aging)
Factory Method	Creational	Creating different channel implementations (Shopify, etc.)
Command	Behavioral	Encapsulating price-change requests with undo/redo support
Decorator	Structural	Adding audit logging to pricing commands dynamically
Observer	Behavioral	Triggering repricing when inventory levels change
Adapter	Structural	Adapting demand signal data to domain model
Template Method	Behavioral	Defining repricing workflow skeleton (fetch → validate → execute → log)

---

6. Tech Stack

- **Language:** Java (JDK 21 or later)
- **Build Tool:** Maven 3.8+
- **IDE:** Eclipse or VS Code with Java extensions
- **Version Control:** Git + GitHub (private repo)
- **Testing:** JUnit 5
- **Data Source:** CSV files for inventory and demand signals
- **Logging:** SLF4J + Logback
- **Optional:** Spring Boot for REST API (Milestone 3)

---

7. Functionalities by End of Milestone 2

Core Pattern Implementation

- **Inventory (Singleton)** with thread-safe stock tracking and observer notification
- **SalesChannel interface** with Shopify implementation
- **PricingStrategy interface** with 3 concrete strategies:
  - CostPlusUpMarkupStrategy
  - CompetitorAwarePricingStrategy
  - InventoryAgingStrategy

- **PricingCommand** with execute/undo support and full history
- **AuditLogger (Decorator)** wrapping all price updates with compliance logging
- **Observer pattern** wired so inventory changes trigger repricing checks

NEW: Lightweight AI + Decision Logic

- **DemandForecaster** using simple moving average (7-day vs 30-day)
  - Detects trend: RISING (>10% growth), FALLING (<-10% decay), STABLE
  - No external ML library—pure statistical logic
- **PricingStrategySelector** that dynamically picks strategy based on demand trend
  - RISING demand → CompetitorAwarePricingStrategy (aggressive positioning)
  - STABLE demand → CostPlusUpMarkupStrategy (margin optimization)
  - FALLING demand → InventoryAgingStrategy (clear old stock)

NEW: Shopify API Integration

- **ShopifyApiClient** with GraphQL support
  - Fetch live product data and inventory
  - Update prices in real-time via mutation
  - Mock-based tests (no real credentials needed)
- **ShopifyChannel** implementing SalesChannel interface
  - Caching with 5-minute TTL
  - Error handling with fallback to cached prices

---

8. Contributions

Person	Module	Responsibility
Aditi Bailur	Inventory & Patterns	Singleton implementation, Observer wiring, core pattern validation

Yashika Lodh	Pricing Intelligence	DemandForecaster, StrategySelector, PricingStrategy implementations
Pratham Rathod	Command & Audit	PricingCommand, PricingCommandInvoker, AuditLogger (Decorator), undo/redo
Sai Vinayaka Venkata Prateek Kacham	Channels & Integration	ShopifyApiClient, ShopifyChannel, error handling, caching logic

9. GitHub Repository

https://github.com/yashika-lodh/CSYE6300_Group_1.git